휴리스틱 알고리즘

HEURISTIC ALGORITHMS

네 가지 공개 코딩 문제로 배우는 알고리즘

공개 학습 문제는 네 개입니다

6~11장, 챕터별 4문제씩 총 24문제 중 기말에 10문제 출제

번호	문제	난도
11-1	격자에서 맨해튼 거리	아주 쉬움
11-2	가장 가까운 도시부터 방문	쉬움
11-3	담금질 기법의 한 단계 수락 판단	기본
11-4	한 번의 도시 교환으로 순회 개선	조금 도전

기말은 별도 객관식·서술형 없이 코드 실행 결과로 평가합니다.

빠른 선택과 조금씩 고치는 탐색

큰 탐색 공간에서 먼저 쓸 만한 답을 만들 수 있습니다.

거리 계산

현재 위치와 목표 사이의 거리를 계산합니다.

탐욕 선택

가까운 미방문 도시를 차례로 고릅니다.

이웃 탐색

현재 경로의 일부를 바꾼 후보들을 비교합니다.

나쁜 후보도 가끔 받아들이는 이유

지역적으로 좋은 해 주변에서 벗어날 기회를 만듭니다.

좋은 후보

비용이 작거나 같으면 받아들입니다.

나쁜 후보

온도와 비용 차이로 계산한 확률에 따라 받아들입니다.

재현 가능한 실습

난수 u를 입력으로 받아 수락 여부 한 단계만 계산합니다.

약 2시간 수업 흐름

한 문제를 이해하고 직접 구현하는 시간을 함께 확보합니다.

시간	내용
0–20분	맨해튼 거리
20–45분	최근접 도시 순회
45–55분	쉬는 시간
55–80분	담금질 한 단계
80–105분	한 번의 위치 교환
105–120분	직접 구현·경계 테스트

마지막에는 빈 코드에서 다시 작성하고 테스트합니다.

11-1 격자에서 맨해튼 거리

아주 쉬움

격자의 두 점 (x1,y1), (x2,y2)가 주어집니다. 각 좌표는 −10,000..10,000의 정수입니다. 한 번에 상하좌우로 한 칸 이동할 수 있고 장애물은 없습니다. 두 점 사이의 최소 이동 횟수 |x1−x2|+|y1−y2|를 반환하세요.

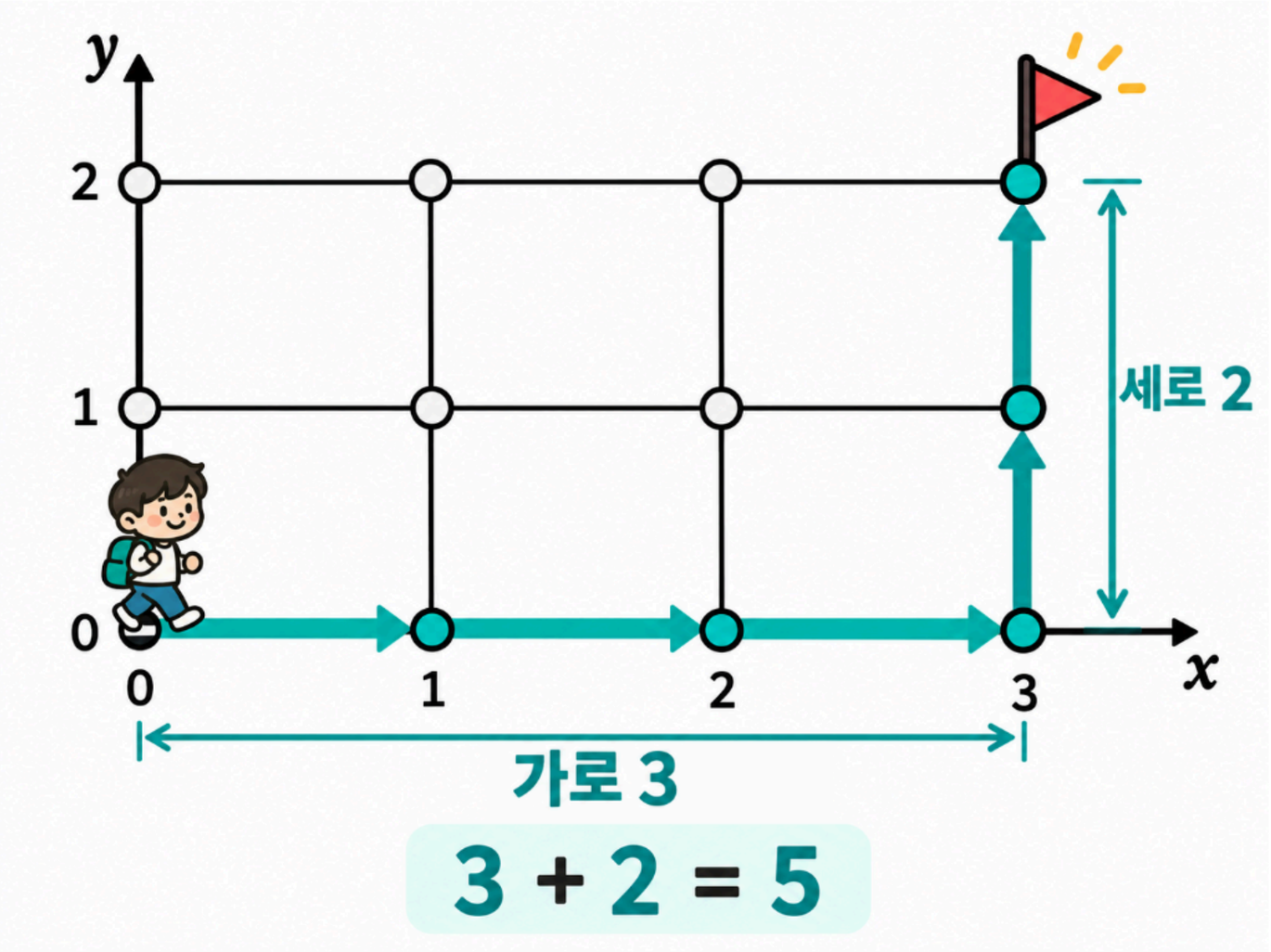
함수와 공개 예제

Python: manhattan(x1, y1, x2, y2)

입력	반환값
0, 0, 3, 2	5
2, 3, 2, 3	0
2, -1, -2, 2	7

int manhattan(int x1, int y1, int x2, int y2)

격자에서 맨해튼 거리의 작은 예



상하좌우 이동
가로 세 칸
세로 두 칸
최소 3+2=5번

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

출발	도착	가로 차이	세로 차이	합
(0,0)	(3,2)	3	2	5
(2,3)	(2,3)	0	0	0
(2,–1)	(–2,2)	4	3	7

가로 차이의 절댓값과 세로 차이의 절댓값을 더합니다. 대각선으로 한 번에 이동하는 규칙은 사용하지 않습니다.

이 문제의 계산 방법

11-1

가로 차이의 절댓값과 세로 차이의 절댓값을 더합니다. 대각선으로 한 번에 이동하는 규칙은 사용하지 않습니다.

좌표 차이가 음수여도 거리는 양수입니다. 두 점이 같으면 이동이 필요 없으므로 0입니다.

격자에서 맨해튼 거리 · Python

```
def manhattan(x1, y1, x2, y2):
    return abs(x1-x2) + abs(y1-y2)
```

시간 O(1), 추가 공간 O(1)

격자에서 맨해튼 거리 · C

```
int manhattan(int x1, int y1, int x2, int y2) {
    int dx=x1-x2, dy=y1-y2;
    if (dx<0) dx=-dx;
    if (dy<0) dy=-dy;
    return dx+dy;
}
```

시간 O(1), 추가 공간 O(1)

경계 입력과 복잡도

11-1

경계 입력

좌표 차이가 음수여도 거리는 양수입니다. 두 점이 같으면 이동이 필요 없으므로 0입니다.

계산량

시간 O(1), 추가 공간 O(1)

직접 작성하고 수정합니다

11-1 격자에서 맨해튼 거리

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

11-2 가장 가까운 도시부터 방문

쉬움

도시 n개(n=1..9)의 이동 비용 행렬 dist가 주어집니다. 대각선은 0, 비용은 0..100이며 모든 도시 사이에 이동할 수 있고 방향에 따라 비용이 다를 수 있습니다. 0번에서 출발해 현재 도시에서 비용이 가장 작은 미방문 도시로 이동합니다. 동률이면 번호가 작은 도시를 고릅니다. 모든 도시를 방문한 뒤 0번으로 돌아오는 이 절차의 총 비용을 반환하세요. n=1의 답은 0입니다.

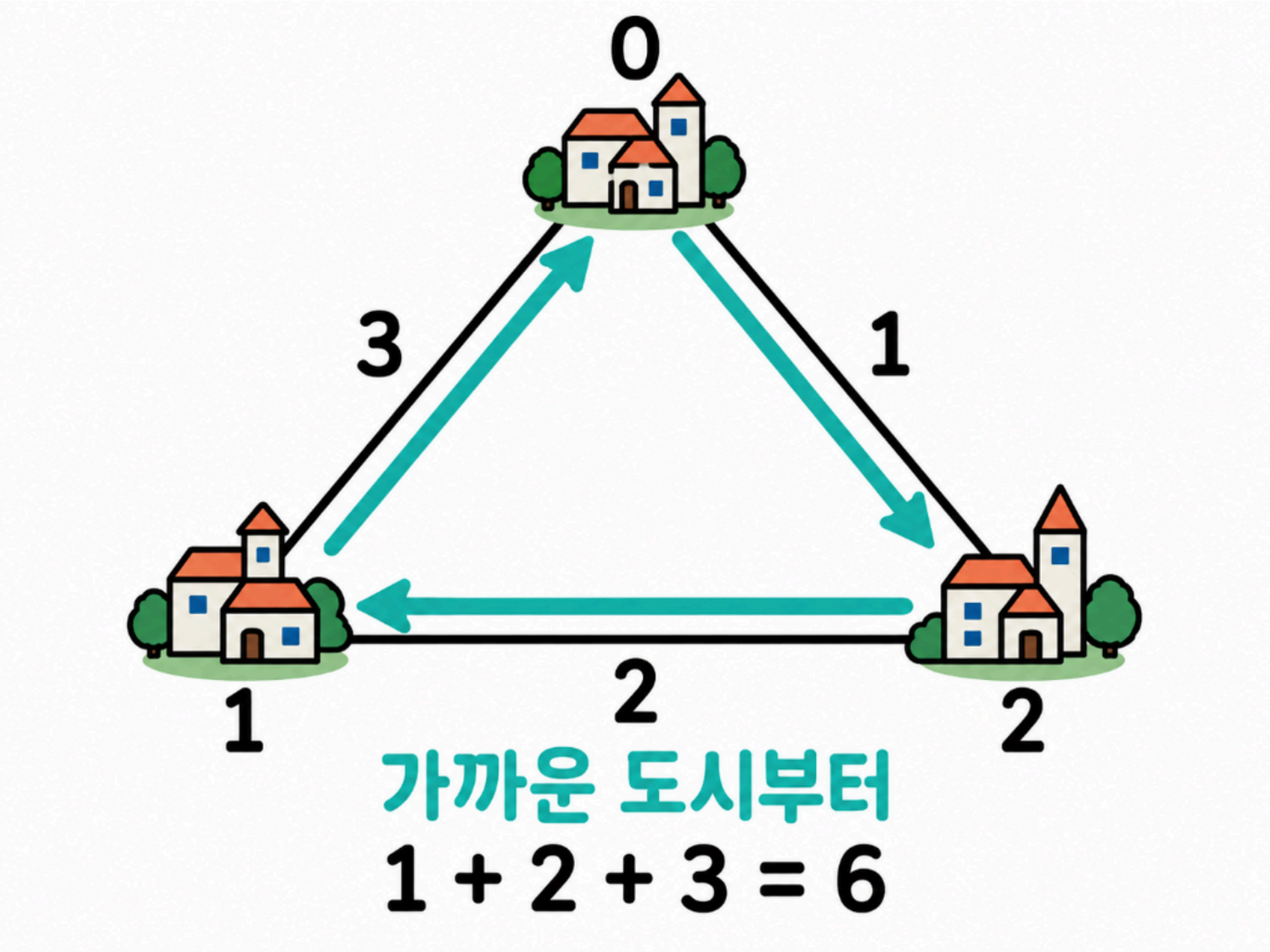
함수와 공개 예제

Python: nearest_neighbor(dist)

입력	반환값
[[0,3,1],[3,0,2],[1,2,0]]	6
[[0]]	0
[[0,1,1],[5,0,1],[2,7,0]]	4

int nearest_neighbor(const int dist[][9], int n)

가장 가까운 도시부터 방문의 작은 예



0에서 가까운 2
다음은 남은 1
마지막은 0으로 귀환
1+2+3=6

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

현재	미방문 후보 비용	선택	누적 비용
0	1: 3, 2: 1	2	1
2	1: 2	1	3
1	출발 도시로: 3	0	6

가까운 미방문 도시 하나만 선택하고 방문 표시합니다. 매번 전체 미방문 후보를 다시 살펴본 뒤 마지막 귀환 비용을 더합니다.

이 문제의 계산 방법

11-2

가까운 미방문 도시 하나만 선택하고 방문 표시합니다. 매번 전체 미방문 후보를 다시 살펴본 뒤 마지막 귀환 비용을 더합니다.

이 절차의 비용을 구하며 전역 최적값을 보장하지 않습니다. 동률은 작은 도시 번호, 이미 방문한 도시는 후보에서 제외합니다.

가장 가까운 도시부터 방문 · Python

```
def nearest_neighbor(dist):
    n = len(dist)
    used = [False]*n
    used[0] = True
    current, total = 0, 0
    for _ in range(n-1):
        nxt = min((v for v in range(n) if not used[v]),
                  key=lambda v: dist[current][v])
        total += dist[current][nxt]
        used[nxt] = True
        current = nxt
    return total + dist[current][0]
```

시간 $O(n^2)$, 방문 배열 $O(n)$

가장 가까운 도시부터 방문 · C

```
int nearest_neighbor(const int dist[][9], int n) {
    int used[9]={0}, current=0, total=0; used[0]=1;
    for (int step=0; step<n-1; ++step) {
        int next=-1;
        for (int v=0; v<n; ++v)
            if (!used[v] && (next<0 || dist[current][v]<dist[current][next]))
                next=v;
        total+=dist[current][next]; used[next]=1; current=next;
    }
    return total+dist[current][0];
}
```

시간 $O(n^2)$, 방문 배열 $O(n)$

경계 입력과 복잡도

11-2

경계 입력

이 절차의 비용을 구하며 전역 최적값을 보장하지 않습니다. 동률은 작은 도시 번호, 이미 방문한 도시는 후보에서 제외합니다.

계산량

시간 $O(n^2)$, 방문 배열 $O(n)$

직접 작성하고 수정합니다

11-2 가장 가까운 도시부터 방문

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

10분 쉬었다가 이어갑니다

앞의 두 계산을 정리합니다.

뒤의 문제는 앞에서 익힌 계산을
반복하거나 탐색에 연결합니다.

11-3 담금질 기법의 한 단계 수락 판단

기본

비용을 최소화합니다. 현재 비용 current와 새 후보 비용 candidate는 0..10,000의 정수, 온도 temp는 0..1000의 실수, 주어진 난수 u는 0 이상 1 미만의 실수입니다. 후보 비용이 현재 이하이면 항상 1을 반환합니다. 후보가 더 나쁘고 temp=0이면 0입니다. 그 외에는 u < exp((current−candidate)/temp)이면 1, 아니면 0을 반환하세요. 난수를 새로 만들지 말고 주어진 u를 사용합니다.

함수와 공개 예제

Python: accept_move(current, candidate, temp, u)

입력	반환값
10, 8, 0, 0.9	1
10, 12, 2, 0.2	1
10, 12, 2, 0.8	0

int accept_move(int current, int candidate, double temp, double u)

담금질 기법의 한 단계 수락 판단의 작은 예



비용 10에서 12로
온도는 2
수락 확률은 약 0.368
u=0.2면 수락합니다.

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

current	candidate	temp	u	판단
10	8	0	0.9	개선 → 수락
10	12	2	0.2	0.2 < 0.368 → 수락
10	12	2	0.8	0.8 ≥ 0.368 → 거부
10	11	0	0	더 나쁨·온도0 → 거부

좋거나 같은 후보는 바로 수락합니다. 더 나쁜 후보는 온도와 비용 차이로 확률을 계산하고 주어진 u와 비교합니다.

이 문제의 계산 방법

11-3

좋거나 같은 후보는 바로 수락합니다. 더 나쁜 후보는 온도와 비용 차이로 확률을 계산하고 주어진 u와 비교합니다.

temp=0을 나눗셈 전에 처리합니다. 양의 온도에서 u=0이면 수락하며, 아주 작은 확률의 반올림도 처리합니다. u가 확률과 같으면 거부합니다.
시험에서는 같은 입력이 같은 결과를 내도록 u를 인자로 줍니다.

담금질 기법의 한 단계 수락 판단 · Python

```
import math
def accept_move(current, candidate, temp, u):
    if candidate <= current: return 1
    if temp == 0: return 0
    if u == 0: return 1
    probability = math.exp((current-candidate)/temp)
    return int(u < probability)
```

기본 수학 연산을 상수 시간으로 볼 때 시간 O(1), 공간 O(1)

담금질 기법의 한 단계 수락 판단 · C

```
#include <math.h>
int accept_move(int current, int candidate, double temp, double u) {
    if (candidate<=current) return 1;
    if (temp==0) return 0;
    if (u==0) return 1;
    double probability=exp((current-candidate)/temp);
    return u<probability;
}
```

기본 수학 연산을 상수 시간으로 볼 때 시간 O(1), 공간 O(1)

경계 입력과 복잡도

11-3

경계 입력

temp=0을 나눗셈 전에 처리합니다. 양의 온도에서 u=0이면 수락하며, 아주 작은 확률의 반올림도 처리합니다. u가 확률과 같으면 거부합니다. 시험에서는 같은 입력이 같은 결과를 내도록 u를 인자로 줍니다.

계산량

기본 수학 연산을 상수 시간으로 볼 때 시간 O(1), 공간 O(1)

직접 작성하고 수정합니다

11-3 담금질 기법의 한 단계 수락 판단

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

11-4 한 번의 도시 교환으로 순회 개선

조금 도전

도시 n개(n=1..9)의 완전 이동 비용 행렬 dist가 주어집니다. 대각선은 0, 비용은 0..100이며 방향에 따라 비용이 다를 수 있습니다. route는 모든 도시 번호를 한 번씩 포함하고 첫 값은 0입니다. 첫 도시를 고정하고 나머지 위치 두 곳을 한 번 맞바꾸는 모든 후보를 비교하세요. 원래 경로도 후보에 포함합니다. 각 경로의 마지막에서 0으로 돌아오는 비용까지 더한 최솟값을 반환하세요. 여러 번 연속 교환하는 문제는 아닙니다.

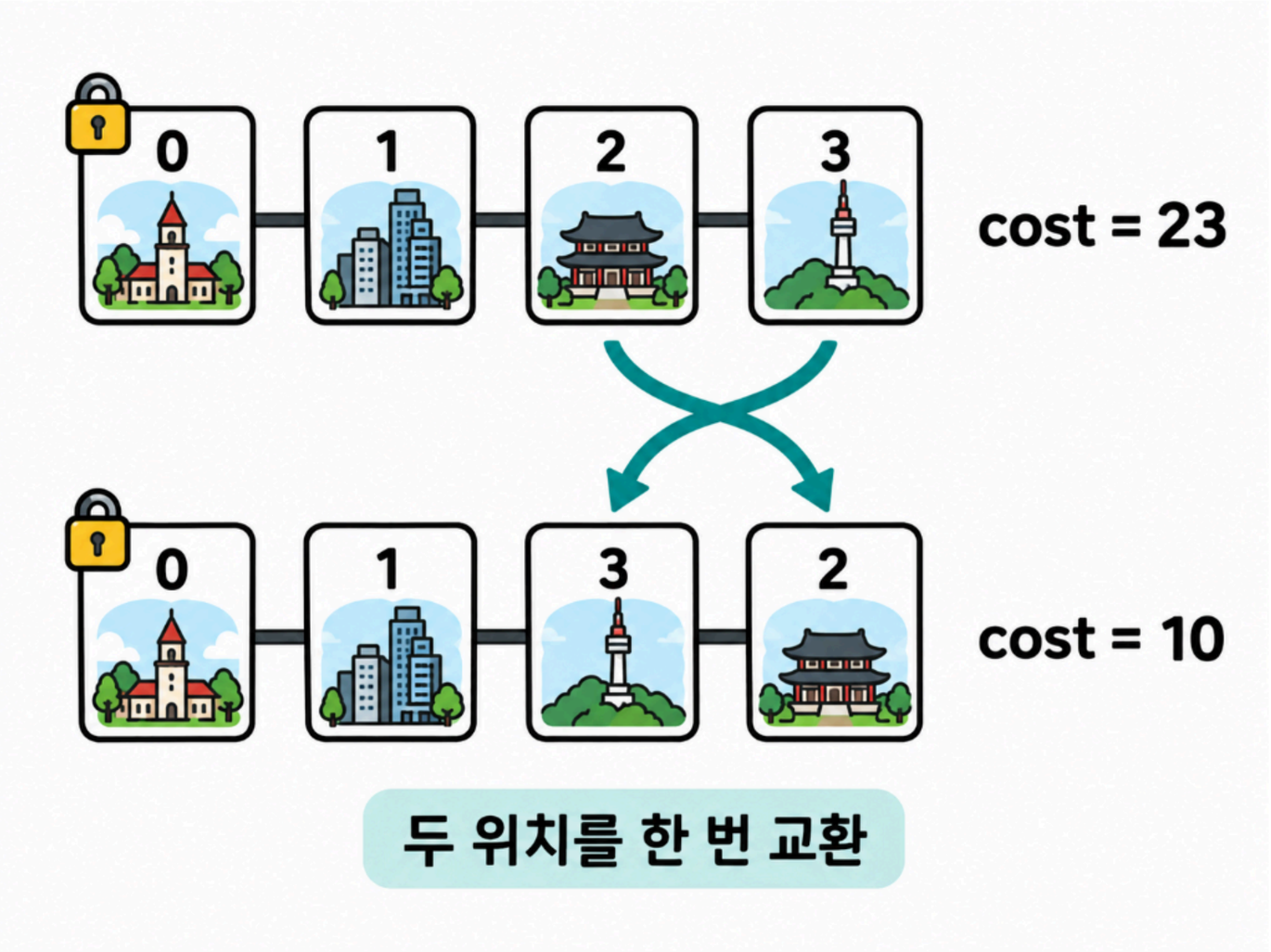
함수와 공개 예제

Python: best_swap_cost(dist, route)

입력	반환값
[[0,3,1,8],[3,0,8,2],[1,8,0,4],[8,2,4,0]], [0,1,2,3]	10
[[0]], [0]	0
[[0,5],[8,0]], [0,1]	13

int best_swap_cost(const int dist[][9], int n, const int route[])

한 번의 도시 교환으로 순회 개선의 작은 예



첫 도시 0은 고정
2와 3의 위치 교환
원래 비용 23에서
비용 10으로 개선

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

교환한 위치	경로	귀환 포함 비용
교환 안 함	0,1,2,3	23
1,2	0,2,1,3	19
1,3	0,3,2,1	23
2,3	0,1,3,2	10
최솟값		10

원래 비용으로 best를 시작합니다. 첫 위치를 제외한 모든 두 위치를 교환하고 비용을 계산한 뒤 원래 순서로 되돌립니다.

이 문제의 계산 방법

11-4

원래 비용으로 best를 시작합니다. 첫 위치를 제외한 모든 두 위치를
교환하고 비용을 계산한 뒤 원래 순서로 되돌립니다.

교환할 위치 쌍이 없으면 원래 비용을 반환합니다. 매 후보 뒤에 복구하며,
한 번의 교환으로 닿는 이웃 중 최선을 구합니다.

한 번의 도시 교환으로 순회 개선 · Python

```
def best_swap_cost(dist, route):
    route = route[:]
    n = len(route)
    def cost():
        return sum(dist[route[i]][route[(i+1)%n]] for i in range(n))
    best = cost()
    for i in range(1, n):
        for j in range(i+1, n):
            route[i], route[j] = route[j], route[i]
            best = min(best, cost())
            route[i], route[j] = route[j], route[i]
    return best
```

위치 쌍 $O(n^2)$ × 경로 비용 $O(n)$, 시간 $O(n^3)$, 복사 배열 $O(n)$

한 번의 도시 교환으로 순회 개선 · C

```
static int swap_route_cost(const int d[][9], int n, const int r[]) {
    int sum=0;
    for (int i=0; i<n; ++i) sum+=d[r[i]][r[(i+1)%n]];
    return sum;
}
int best_swap_cost(const int dist[][9], int n, const int route[]) {
    int r[9]; for (int i=0; i<n; ++i) r[i]=route[i];
    int best=swap_route_cost(dist,n,r);
    for (int i=1; i<n; ++i) for (int j=i+1; j<n; ++j) {
        int t=r[i]; r[i]=r[j]; r[j]=t;
        int value=swap_route_cost(dist,n,r);
        if (value<best) best=value;
        t=r[i]; r[i]=r[j]; r[j]=t;
    }
    return best;
}
```

위치 쌍 $O(n^2)$ × 경로 비용 $O(n)$, 시간 $O(n^3)$, 복사 배열 $O(n)$

경계 입력과 복잡도

11-4

경계 입력

교환할 위치 쌍이 없으면 원래 비용을 반환합니다. 매 후보 뒤에 복구하며, 한 번의 교환으로 닿는 이웃 중 최선을 구합니다.

계산량

위치 쌍 $O(n^2)$ × 경로 비용 $O(n)$, 시간 $O(n^3)$, 복사 배열 $O(n)$

직접 작성하고 수정합니다

11-4 한 번의 도시 교환으로 순회 개선

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

이 장의 네 문제 정리

강의노트와 사이트는 같은 함수 규격을 사용합니다.

문제	함수	난도
11-1	manhattan	아주 쉬움
11-2	nearest_neighbor	쉬움
11-3	accept_move	기본
11-4	best_swap_cost	조금 도전

공개 문제의 조건 안에서 올바른 반환값을 구합니다.

기말 준비

공개 24문제 중 10문제 출제

6~11장, 챕터마다 네 문제를 공부합니다.
정해진 함수 인자와 반환값을 지킵니다.
직접 작성한 코드의 실행 결과로 평가합니다.

휴리스틱 알고리즘 정리

HEURISTIC ALGORITHMS

11-1 격자에서 맨해튼 거리
11-2 가장 가까운 도시부터 방문
11-3 담금질 기법의 한 단계 수락 판단
11-4 한 번의 도시 교환으로 순회 개선